МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу «Операционные системы»

Выполнил: Яковлев А. С.
Группа: М8О-203БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: приобретение практических навыков в

- управлении потоками в ОС;
- обеспечении синхронизации между потоками.

Задание: составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы. Продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. Привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Вариант: 9. Рассчитать детерминант матрицы (используя определение детерминанта).

Метод решения

Определитель по определению

Определитель квадратной матрицы $A = (a_{ij})$ порядка n по определению равен

$$\det A = \sum_{\sigma \in S_n} \operatorname{sgn}(\sigma)\, a_{0,\sigma(0)}\, a_{1,\sigma(1)} \cdots a_{n-1,\sigma(n-1)},$$

где сумма берётся по всем $n!$ перестановкам σ множества $\{0, \ldots, n-1\}$. Каждое слагаемое — произведение n элементов, взятых по одному из каждой строки и каждого столбца. Знак перестановки определяется числом инверсий — пар $i < j$, для которых $\sigma(i) > \sigma(j)$: при чётном числе инверсий $\operatorname{sgn}(\sigma) = +1$, при нечётном — -1.

Перестановки перебираются рекурсивно: функция `go` выбирает для очередной строки свободный столбец, помечает его занятым, домножает накопленное произведение на выбранный элемент и переходит к следующей строке. Число инверсий накапливается по ходу: при выборе столбца c в строке k новых инверсий столько, сколько уже занятых столбцов больше c (они стоят в перестановке раньше и правее). Когда заполнены все строки, произведение возвращается со знаком, определяемым чётностью числа инверсий.

Распараллеливание

Слагаемые суммы независимы, поэтому работу можно разделить между потоками без обмена данными по ходу вычислений. Вся сумма разбита на n^2 задач: задача с номером t фиксирует столбцы первых двух строк $c_0 = \lfloor t/n \rfloor$, $c_1 = t \bmod n$ и перебирает все перестановки с таким началом. Задачи с $c_0 = c_1$ пусты, остальные $n(n-1)$ задач содержат по $(n-2)!$ слагаемых. Каждая перестановка попадает ровно в одну задачу, поэтому сумма результатов всех задач равна определителю. Разбиение по двум строкам, а не по одной, даёт мелкие задачи (при $n = 12$ — 132 непустые), которые равномернее распределяются по потокам.

Задачи раздаются динамически: общий счётчик `next_task` указывает номер следующей свободной задачи. Освободившийся поток под мьютексом берёт номер и увеличивает счётчик. Так потоки, которые планировщик ОС выполняет медленнее (например, на энергоэффективных ядрах), берут меньше задач и не задерживают остальных.

Каждый поток накапливает сумму своих задач в локальной переменной и по окончании работы один раз прибавляет её к общему результату под тем же мьютексом.

Ограничение числа потоков

Ключ `-t P` задаёт общее число одновременно работающих потоков, включая главный. Программа создаёт $P-1$ потоков функцией `pthread_create`, а главный поток выполняет ту же функцию `worker` наравне с ними. При $P = 1$ дополнительных потоков не создаётся. Затем главный поток ожидает завершения остальных (`pthread_join`).

Описание программы

Структура проекта

Файл	**Содержимое**
`CMakeLists.txt`	сборка: библиотека `detlib` и программа `det`, подключение pthreads
`src/main.c`	разбор ключей запуска, получение матрицы, замер времени, вывод результата
`src/matrix.c`, `.h`	чтение матрицы из файла, генерация случайной матрицы, печать
`src/det.c`, `.h`	вычисление определителя, управление потоками и синхронизация
`tests/m1..m4.txt`	тестовые матрицы с известным определителем

Матрица $n \times n$ хранится в одномерном массиве `double` по строкам: элемент a_{ij} находится по индексу `i * n + j`.

Основные функции

- `double *matrix_read(const char *filename, int *n)` — читает матрицу из файла, возвращает массив или `NULL` при ошибке.
- `double *matrix_random(int n)` — создаёт матрицу из случайных целых чисел от -9 до 9. Генератор инициализируется константой (`srand(1)`), поэтому при одинаковом n матрица всегда одна и та же, что делает замеры сопоставимыми.
- `double go(int row, int *used, double prod, int inv)` — рекурсивно перебирает перестановки для строк начиная с `row`, возвращает сумму соответствующих слагаемых.
- `double do_task(int t)` — фиксирует столбцы первых двух строк по номеру задачи и вызывает `go` со строки 2.
- `void *worker(void *arg)` — функция потока: в цикле берёт задачи из общей очереди, накапливает локальную сумму, в конце добавляет её к результату.
- `double det_calc(double *a, int n, int threads)` — инициализирует общие данные, создаёт $P-1$ потоков, выполняет `worker` в главном потоке, дожидается остальных и возвращает определитель.

Ключи запуска

`-t P` число потоков (обязательный, от 1 до 256)
`-f F` прочитать матрицу из файла `F`
`-n N` сгенерировать случайную матрицу $N \times N$ ($N \leq 15$)
`-p` вывести матрицу
Программа печатает свой PID, определитель и время вычисления.

Использованные источники

1. Таненбаум Э., Бос Х. Современные операционные системы. — СПб.:

Результаты

Конфигурация

Процессор:	M3 Pro
Ядер всего:	11
ОС:	MacOS

Проверка корректности

Программа запускалась на матрицах с заранее известным определителем при разном числе потоков. Ожидаемые значения.

Матрица	**Ожидается**	$P = 1$	$P = 2$	$P = 4$	$P = 8$
`m1.txt` (1×1)	7	7	7	7	7
`m2.txt` (2×2)	-14	-14	-14	-14	-14
`m3.txt` (3×3)	-306	-306	-306	-306	-306
`m4.txt` (4×4)	30	30	30	30	30

Результат не зависит от числа потоков, что подтверждает отсутствие гонок данных: каждая задача вычисляется ровно один раз, а общий результат изменяется только под мьютексом.

Демонстрация числа потоков

Программа была запущена командой `./det -t 4 -n 13`, во время вычислений число её потоков просмотрено стандартными средствами ОС. Получено 4 потока: главный и три созданных.

```
artem@MacBook-Pro-Artem-3 lab3 % ps -M -p 45502
USER     PID   TT   %CPU STAT PRI     STIME     UTIME COMMAND
artem 45502 s000  100.0 R    31T   0:00.01   0:04.50 ./cmake-build-
   debug/det -t 4 -n 13
      45502        100.0 R    31T   0:00.01   0:04.49
      45502        100.0 R    31T   0:00.01   0:04.49
      45502        100.0 R    31T   0:00.01   0:04.49
```

Исследование ускорения и эффективности

Замеры выполнялись на случайных целочисленных матрицах порядка $n = 8 \dots 12$ (одинаковых для всех P) при числе потоков $P = 1, 2, 4, 8, 16$. Каждый замер повторялся 5 раз, в таблицу занесена медиана.

Ускорение $S(P) = T_1/T_P$, эффективность $E(P) = S(P)/P \cdot 100\%$, где T_1 — время при одном потоке, T_P — при P потоках.

Время T_P, с

P	$n = 8$	$n = 9$	$n = 10$	$n = 11$	$n = 12$
1	0.000714	0.006422	0.0665	0.7614	9.696
2	0.000399	0.003333	0.0341	0.3911	4.849
4	0.000236	0.001750	0.0179	0.2024	2.487
8	0.000204	0.001220	0.0118	0.1396	1.791
16	0.000216	0.001140	0.0107	0.1193	1.558

Ускорение $S(P)$

P	$n = 8$	$n = 9$	$n = 10$	$n = 11$	$n = 12$
1	1.00	1.00	1.00	1.00	1.00
2	1.79	1.93	1.95	1.95	2.00
4	3.03	3.67	3.72	3.76	3.90
8	3.50	5.26	5.66	5.45	5.41
16	3.31	5.63	6.22	6.38	6.22

Эффективность $E(P)$, %

P	$n = 8$	$n = 9$	$n = 10$	$n = 11$	$n = 12$
1	100.0	100.0	100.0	100.0	100.0
2	89.5	96.3	97.6	97.3	100.0
4	75.6	91.7	93.0	94.0	97.5
8	43.8	65.8	70.8	68.2	67.7
16	20.7	35.2	38.9	39.9	38.9

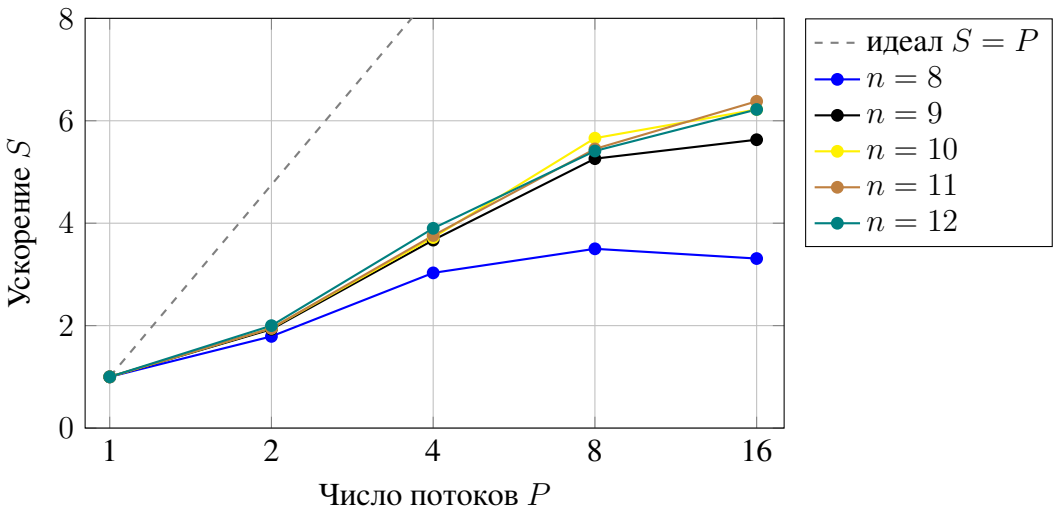


Рис. 1. Зависимость ускорения от числа потоков

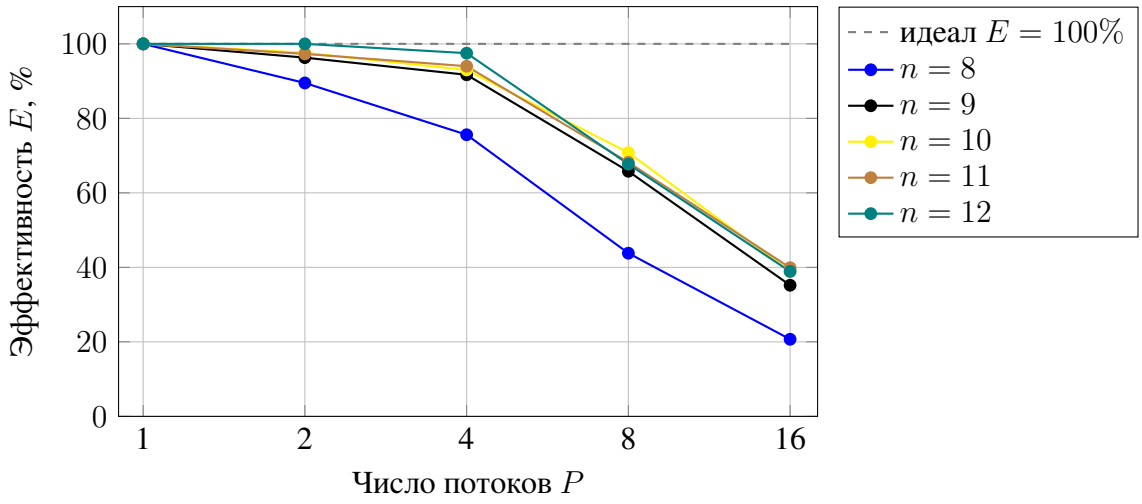


Рис. 2. Зависимость эффективности от числа потоков

Выводы

В ходе работы была написана программа на языке Си, вычисляющая определитель матрицы по определению — как сумму $n!$ слагаемых по всем перестановкам — в многопоточном режиме с использованием POSIX Threads. Работа разбивается на n^2 независимых задач, которые потоки берут из общей очереди; доступ к общим данным (счётчику задач и результату) защищён мьютексом. Число потоков ограничивается ключом запуска с учётом главного потока, что подтверждено средствами ОС. Корректность проверена на матрицах с известным определителем: результат не зависит от числа потоков.

Исследование на процессоре Apple M3 Pro (5 производительных и 6 энергоэффективных ядер) показало следующее.

1. **До 4 потоков ускорение почти линейное.** При $n = 12$ получено $S(2) = 2{,}00$ и $S(4) = 3{,}90$ (эффективность 100% и 97,5%). Задачи независимы, мьютекс захватывается лишь на мгновение при выдаче задачи, а матрица (около 1 КБ) целиком помещается в кэш каждого ядра, поэтому задача ограничена только вычислениями. Все потоки при этом работают на производительных ядрах.

2. **Неоднородные ядра.** При $P = 8$ ускорение растёт заметно слабее ($S = 5{,}41$, $E = 67{,}7\%$ при $n = 12$): производительных ядер только 5, и остальные потоки выполняются на более медленных энергоэффективных ядрах.

3. **Потоков больше, чем ядер.** При $P = 16$, что больше 11 ядер, ускорение выходит на плато: $S \approx 6{,}2$–$6{,}4$ для $n \geq 10$. Это близко к предельной производительности всех ядер процессора. Лишние потоки лишь делят процессорное время, поэтому эффективность падает до 35–40%.

4. **Малые матрицы.** При $n = 8$ (40 320 слагаемых) всё вычисление занимает около 0,7 мс, и затраты на создание и завершение потоков становятся заметными. Поэтому эффективность ниже уже при $P = 4$ (75,6%), ускорение не превышает 3,5, а при $P = 16$ программа даже немного замедляется по сравнению с $P = 8$ (0,216 мс против 0,204 мс). По закону Амдала доля последовательной части (создание потоков, ожидание их завершения) здесь велика. С ростом n эта доля уменьшается, и эффективность растёт.

Таким образом, задача хорошо распараллеливается: независимые слагаемые и минимальная синхронизация дают почти линейное ускорение, пока потоки работают на производительных ядрах, а его предел определяется числом и типом ядер процессора и накладными расходами на потоки при малом объёме работы.